

3CSD_Group 8_Implementation

May 17, 2026

0.1 3CSD Group 8 - GTD-FocalResNet: A Residual Focal-Loss Neural Model for Attack Success Prediction

Members: 1. Abelardo, Aasen Sofia P. 2. Aquino, Matthew Benedict U. 3. Balingit, Den Mar F. 4. Gumban, Joevanni Paulo T.

```
[12]: # Imports
print("Running Cell 1: imports...", flush=True)

from pathlib import Path
import os, random, joblib, warnings, time, zipfile, urllib.request, urllib.error

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import sparse

from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, ConfusionMatrixDisplay
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier

print("Importing optional model libraries...", flush=True)
try:
    from xgboost import XGBClassifier
    print("XGBoost loaded.", flush=True)
except Exception as e:
    XGBClassifier = None
    print(f"XGBoost not available: {e}", flush=True)
```

```

try:
    import torch
    from torch import nn
    print("PyTorch loaded.", flush=True)
except Exception as e:
    torch, nn = None, None
    print(f"PyTorch not available: {e}", flush=True)

print("Imports loaded.", flush=True)

```

Running Cell 1: imports...
 Importing optional model libraries...
 XGBoost loaded.
 PyTorch loaded.
 Imports loaded.

```

[13]: # 2. Configs
print("Running Cell 2: configs...", flush=True)

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
RANDOM_STATE = 42
random.seed(RANDOM_STATE); np.random.seed(RANDOM_STATE)

# Main experiment switches for controlling the flow.
SPLIT_MODE = "temporal"           # "temporal" or "random"
TRAINING_PROFILE = "max"          # "submission" for faster reproducible run,
    ↳ "max" for heavier full-potential run
USE_SAMPLE = False                # True for quick checking only
SAMPLE_NROWS = 5000
CACHE_DATA = True                 # Saves a cleaned .pkl copy so reruns avoid
    ↳ slow Excel loading
SAVE_PROCESSED_SPLITS = True      # Saves train/validation/test splits under
    ↳ data/processed/<split_mode>/
REBUILD_DATA_CACHE = False        # Set True only when raw files/features
    ↳ change
DOWNLOAD_DATA_IF_MISSING = False  # Set True to try downloading raw GTD files
    ↳ if data/raw is empty
GTD_MAIN_URL = os.getenv("GTD_MAIN_URL", "https://www.start.umd.edu/system/
    ↳ files/globalterrorismdb_0522dist.xlsx") # Optional official/direct URL
    ↳ supplied by user
GTD_SUPP_URL = os.getenv("GTD_SUPP_URL", "https://www.start.umd.edu/system/
    ↳ files/globalterrorismdb_2021Jan-June_1222dist.xlsx") # Optional official/
    ↳ direct URL supplied by user
USE_GPU = True
RUN_LEARNING_CURVES = True
RUN_FEATURE_IMPORTANCE = True

```

```

HISTORY_SAMPLE_N = 20000          # max rows used to compute per-epoch train/
    ↪validation metrics
HISTORY_SMOOTH_WINDOW = 3        # rolling mean window for cleaner displayed
    ↪curves

if TRAINING_PROFILE == "max":
    # Heavier settings that use more of the dataset and train models longer.
    # Use this for final result generation if runtime is acceptable.
    USE_SAMPLE = False
    MODEL_EPOCHS = 60
    CURVE_EPOCHS = 20
    XGB_N_ESTIMATORS = 800
    XGB_MAX_DEPTH = 5
    LEARNING_CURVE_FRACTIONS = [0.10, 0.25, 0.50, 0.75, 1.00]
    LEARNING_CURVE_MAX_TRAIN_N = None    # None means no cap; uses all
    ↪training rows for the 1.00 point
    PERMUTATION_SAMPLE_N = 10000
    N_PERM_REPEATS = 5
else:
    # Lighter settings for notebook submission and reruns.
    MODEL_EPOCHS = 25
    CURVE_EPOCHS = 10
    XGB_N_ESTIMATORS = 300
    XGB_MAX_DEPTH = 4
    LEARNING_CURVE_FRACTIONS = [0.25, 0.50, 1.00]
    LEARNING_CURVE_MAX_TRAIN_N = 60000
    PERMUTATION_SAMPLE_N = 3000
    N_PERM_REPEATS = 2

print("Cell 2 complete: configs set.", flush=True)
print("Split mode:", SPLIT_MODE)
print("Training profile:", TRAINING_PROFILE)
print("Download if missing:", DOWNLOAD_DATA_IF_MISSING)

```

Running Cell 2: configs...

Cell 2 complete: configs set.

Split mode: temporal

Training profile: max

Download if missing: False

```

[14]: # 3. Remaining setup and helper functions
print("Running Cell 3: paths and helper functions...", flush=True)

# Paths for clearer location keepings
ROOT = Path.cwd().parent if Path.cwd().name.lower() == "notebooks" else Path.
    ↪cwd()

```

```

RAW = ROOT / "data" / "raw" # This path is for the raw dataset files and can be
    ↪changable.
PROCESSED = ROOT / "data" / "processed"
OUT = ROOT / "results" / "light_reproducible" / SPLIT_MODE
FIG = OUT / "figures"; TAB = OUT / "tables"; MOD = ROOT / "models" /
    ↪"light_reproducible" / SPLIT_MODE
for d in [FIG, TAB, MOD, PROCESSED]: d.mkdir(parents=True, exist_ok=True)

TARGET = "success"; ID = "eventid"
NUM =
    ↪["iyear", "imonth", "iday", "extended", "latitude", "longitude", "specificity", "vicinity", "doubtt
CAT =
    ↪["region_txt", "country_txt", "attacktype1_txt", "targettype1_txt", "targetsubtype1_txt", "natlty1_t
LEAKAGE_EXCLUDED =
    ↪["nkill", "nwound", "property", "ransom", "hostkidoutcome", "nreleased"]

# This area is assisted by AI for cleaning the dataset and engineering
    ↪features, but the modeling and analysis are entirely human-driven.
# The code is designed to be clear and reproducible, with a focus on simplicity
    ↪and interpretability rather than raw performance.
# The dataset is loaded from the "raw" directory, processed, and the results
    ↪are saved in the "results" directory, organized by split mode.
def is_valid_data_file(path):
    """Basic validation so interrupted downloads are not mistaken for real GTD
    ↪files."""
    path = Path(path)
    if not path.exists() or path.stat().st_size == 0:
        return False
    if path.suffix.lower() == ".xlsx":
        return zipfile.is_zipfile(path)
    return True

def resolve_file(stems):
    for stem in stems:
        for ext in [".xlsx", ".xls", ".csv", ".parquet"]:
            p = RAW / f"{stem}{ext}"
            if is_valid_data_file(p):
                return p
        raise FileNotFoundError(f"Missing valid file for one of {stems} in {RAW}")

def raw_file_exists(stems):
    try:
        resolve_file(stems)
        return True
    except FileNotFoundError:
        return False

```

```

def download_file(url, destination, retries=3, chunk_size=1024*1024):
    """Download with retries and a .part file to avoid corrupted partial .xlsx
    ↪ files."""
    destination = Path(destination)
    destination.parent.mkdir(parents=True, exist_ok=True)
    part = destination.with_suffix(destination.suffix + ".part")

    # Remove old incomplete files from previous failed attempts.
    if part.exists():
        part.unlink()
    if destination.exists() and not is_valid_data_file(destination):
        print(f"Removing incomplete/corrupted file: {destination}")
        destination.unlink()

    for attempt in range(1, retries + 1):
        try:
            print(f"Downloading {destination.name} (attempt {attempt}/
            ↪ {retries}) ...", flush=True)
            with urllib.request.urlopen(url, timeout=60) as response,
            ↪ open(part, "wb") as f:
                total = int(response.headers.get("Content-Length") or 0)
                downloaded = 0
                while True:
                    block = response.read(chunk_size)
                    if not block:
                        break
                    f.write(block)
                    downloaded += len(block)
                    if total:
                        print(f" {downloaded/1024/1024:.1f} MB / {total/1024/
            ↪ 1024:.1f} MB", end="\r", flush=True)
                        print()
                    if total and downloaded < total:
                        raise urllib.error.ContentTooShortError(
                            f"retrieval incomplete: got only {downloaded} out of
            ↪ {total} bytes",
                            (str(part), None)
                        )
                part.replace(destination)
                if not is_valid_data_file(destination):
                    destination.unlink(missing_ok=True)
                    raise ValueError(f"Downloaded file is not a valid {destination.
            ↪ suffix} file.")
                print(f"Saved: {destination}")
                return destination

```

```

except Exception as e:
    print(f"Download failed: {e}")
    if part.exists():
        part.unlink()
    if attempt == retries:
        raise
    time.sleep(3 * attempt)

def get_cache_path():
    cache_name = f"gtd_light_{'sample_' + str(SAMPLE_NROWS) if USE_SAMPLE else 'full'}.pkl"
    return PROCESSED / cache_name

def verify_raw_data_available():
    main_ok = raw_file_exists(["globalterrorismdb_0522dist",
    ↪ "globalterrorismdb", "gtd"])
    supp_ok = raw_file_exists(["globalterrorismdb_2021Jan-June_1222dist",
    ↪ "gtd_2021Jan-June", "gtd_2021"])
    missing = []
    if not main_ok:
        missing.append("main GTD file")
    if not supp_ok:
        missing.append("2021 supplement file")
    if missing:
        raise FileNotFoundError(
            f"Missing {'', '.join(missing)} in {RAW}. Run the download cell with
    ↪ DOWNLOAD_DATA_IF_MISSING=True, "
            "or manually place the GTD Excel/CSV files in data/raw/."
        )
    print("Raw GTD files are available in:", RAW)

def download_raw_data():
    main_ok = raw_file_exists(["globalterrorismdb_0522dist",
    ↪ "globalterrorismdb", "gtd"])
    supp_ok = raw_file_exists(["globalterrorismdb_2021Jan-June_1222dist",
    ↪ "gtd_2021Jan-June", "gtd_2021"])
    if main_ok and supp_ok:
        print("Raw files already exist; skipping download.")
        return
    if not GTD_MAIN_URL and not main_ok:
        raise ValueError("GTD_MAIN_URL is empty. Add a direct official URL or
    ↪ place the file in data/raw/.")
    if not GTD_SUPP_URL and not supp_ok:
        raise ValueError("GTD_SUPP_URL is empty. Add a direct official URL or
    ↪ place the file in data/raw/.")
    if GTD_MAIN_URL and not main_ok:

```

```

        download_file(GTD_MAIN_URL, RAW / "globalterrorismdb_0522dist.xlsx")
    if GTD_SUPP_URL and not supp_ok:
        download_file(GTD_SUPP_URL, RAW /
↳"globalterrorismdb_2021Jan-June_1222dist.xlsx")
        verify_raw_data_available()

def read_table(path, usecols, nrows=None):
    if path.suffix.lower() in [".xlsx", ".xls"]:
        cols = pd.read_excel(path, nrows=0).columns
        return pd.read_excel(path, usecols=[c for c in usecols if c in cols],
↳nrows=nrows)
    if path.suffix.lower() == ".csv":
        cols = pd.read_csv(path, nrows=0).columns
        return pd.read_csv(path, usecols=[c for c in usecols if c in cols],
↳nrows=nrows, low_memory=False)
    df = pd.read_parquet(path, columns=[c for c in usecols if c in pd.
↳read_parquet(path).columns])
    return df.head(nrows) if nrows else df

def load_data():
    n = SAMPLE_NROWS if USE_SAMPLE else None
    cache_path = get_cache_path()

    if CACHE_DATA and cache_path.exists() and not REBUILD_DATA_CACHE:
        print(f"Loading cached cleaned data: {cache_path}")
        return pd.read_pickle(cache_path)

    verify_raw_data_available()
    print("Reading raw GTD files. This is slow only the first time; a cache
↳will be saved after loading.")
    usecols = list(dict.fromkeys([ID] + NUM + CAT + [TARGET]))
    main = resolve_file(["globalterrorismdb_0522dist", "globalterrorismdb",
↳"gtd"])
    supp = resolve_file(["globalterrorismdb_2021Jan-June_1222dist",
↳"gtd_2021Jan-June", "gtd_2021"])
    df = pd.concat([read_table(main, usecols, n), read_table(supp, usecols,
↳n)], ignore_index=True)
    df = df.drop_duplicates(ID).query("success in [0, 1]").
↳reset_index(drop=True)
    for c in [x for x in NUM if x in df]: df[c] = pd.to_numeric(df[c],
↳errors="coerce")
    for c in [x for x in CAT if x in df]: df[c] = df[c].astype("string").
↳fillna("Unknown")

    if CACHE_DATA:
        df.to_pickle(cache_path)

```

```

        print(f"Saved cleaned data cache: {cache_path}")
    return df

def split_data(X, y):
    if SPLIT_MODE == "temporal":
        tr = X.iyear <= 2014; va = (X.iyear > 2014) & (X.iyear <= 2017); te = X.
        ↪iyear > 2017
        return X[tr].copy(), X[va].copy(), X[te].copy(), y[tr].copy(), y[va].
        ↪copy(), y[te].copy()
        X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=0.30,
        ↪stratify=y, random_state=RANDOM_STATE)
        X_va, X_te, y_va, y_te = train_test_split(X_tmp, y_tmp, test_size=0.50,
        ↪stratify=y_tmp, random_state=RANDOM_STATE)
        return X_tr, X_va, X_te, y_tr, y_va, y_te

def onehot():
    try: return OneHotEncoder(handle_unknown="infrequent_if_exist",
    ↪min_frequency=20, sparse_output=True)
    except TypeError: return OneHotEncoder(handle_unknown="ignore", sparse=True)

def preprocessor(num, cat):
    return ColumnTransformer([
        ("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("sc",
        ↪StandardScaler())])), num),
        ("cat", Pipeline([("imp", SimpleImputer(strategy="constant",
        ↪fill_value="Unknown")), ("oh", onehot())])), cat),
    ], sparse_threshold=0.3)

def scores(name, split, y, prob, threshold=0.50):
    pred = (prob >= threshold).astype(int)
    return {"model": name, "split": split, "accuracy": accuracy_score(y, pred),
    ↪"precision": precision_score(y, pred, zero_division=0),
        "recall": recall_score(y, pred, zero_division=0), "f1": f1_score(y,
    ↪pred, zero_division=0),
        "roc_auc": roc_auc_score(y, prob)}

def proba(model, X): return model.predict_proba(X)[: , 1]
setup_summary = pd.DataFrame({
    "setting": [
        "project_root", "raw_dir", "processed_dir", "output_dir",
        "split_mode", "training_profile", "use_sample", "cache_data",
        "rebuild_data_cache", "download_data_if_missing", "cleaned_cache"
    ],
    "value": [
        str(ROOT), str(RAW), str(PROCESSED), str(OUT),
        SPLIT_MODE, TRAINING_PROFILE, USE_SAMPLE, CACHE_DATA,

```



```

        REBUILD_DATA_CACHE, DOWNLOAD_DATA_IF_MISSING, str(get_cache_path())
    ]
})

print("Cell 3 complete: paths and helper functions are ready.", flush=True)
print("Next: run the download/verify cell, then the clean/cache/split cell.",
      flush=True)
display(setup_summary)

```

Running Cell 3: paths and helper functions...

Cell 3 complete: paths and helper functions are ready.

Next: run the download/verify cell, then the clean/cache/split cell.

	setting \	value
0	project_root	c:\Documents\Compisci\CSELEC2C\Project
1	raw_dir	c:\Documents\Compisci\CSELEC2C\Project\data\raw
2	processed_dir	c:\Documents\Compisci\CSELEC2C\Project\data\pro...
3	output_dir	c:\Documents\Compisci\CSELEC2C\Project\results\...
4	split_mode	temporal
5	training_profile	max
6	use_sample	False
7	cache_data	True
8	rebuild_data_cache	False
9	download_data_if_missing	False
10	cleaned_cache	c:\Documents\Compisci\CSELEC2C\Project\data\pro...

0.2 2. Download or verify the raw dataset

Run this cell before cleaning. If a cleaned cache already exists in `data/processed/`, the notebook can skip downloading and raw Excel parsing. If the cache is missing, set `DOWNLOAD_DATA_IF_MISSING = True` in the setup cell to download from the direct GTD links, or manually place the files in `data/raw/`.

```
[15]: cache_path = get_cache_path()
```

```

if CACHE_DATA and cache_path.exists() and not REBUILD_DATA_CACHE:
    print(f"Cleaned cache already exists; raw download is optional:␣
    ↳{cache_path}")
elif DOWNLOAD_DATA_IF_MISSING:
    download_raw_data()
else:
    print("DOWNLOAD_DATA_IF_MISSING=False; checking whether raw files already␣
    ↳exist locally.")
    verify_raw_data_available()

```

Cleaned cache already exists; raw download is optional:

c:\Documents\CompSci\CSELEC2C\Project\data\processed\gtd_light_full.pkl

0.3 3. Clean/cache data and create the selected split

This cell performs the actual data processing step. It loads the cleaned cache from `data/processed/` when available. If no cache exists, it reads the raw GTD files from `data/raw/`, selects the modeling columns, removes duplicate `eventid` rows, filters the binary target, converts numeric fields, fills missing categorical values with "Unknown", saves the cleaned cache, and then creates the selected temporal or random split.

In addition to the cleaned `.pkl` cache, this cell can save the actual train/validation/test split files under `data/processed/<split_mode>/` when `SAVE_PROCESSED_SPLITS = True`.

```

[16]: df = load_data()
num = [c for c in NUM if c in df.columns]
cat = [c for c in CAT if c in df.columns]

X, y = df[num + cat].copy(), df[TARGET].astype(int).copy()

X_train, X_valid, X_test, y_train, y_valid, y_test = split_data(X, y)

if SAVE_PROCESSED_SPLITS:
    split_dir = PROCESSED / SPLIT_MODE
    split_dir.mkdir(parents=True, exist_ok=True)
    X_train.to_parquet(split_dir / "X_train.parquet", index=False)
    X_valid.to_parquet(split_dir / "X_valid.parquet", index=False)
    X_test.to_parquet(split_dir / "X_test.parquet", index=False)
    y_train.to_frame(TARGET).to_csv(split_dir / "y_train.csv", index=False)
    y_valid.to_frame(TARGET).to_csv(split_dir / "y_valid.csv", index=False)
    y_test.to_frame(TARGET).to_csv(split_dir / "y_test.csv", index=False)
    pd.DataFrame({"numeric_features": pd.Series(num)}).to_csv(split_dir /␣
    ↳"numeric_features.csv", index=False)
    pd.DataFrame({"categorical_features": pd.Series(cat)}).to_csv(split_dir /␣
    ↳"categorical_features.csv", index=False)
    print(f"Saved processed {SPLIT_MODE} split files to: {split_dir}")

summary = pd.DataFrame({

```

```

    "split": ["train", "validation", "test"],
    "n": [len(X_train), len(X_valid), len(X_test)],
    "start_year": [int(X_train.iyear.min()), int(X_valid.iyear.min()),
    ↪int(X_test.iyear.min())],
    "end_year": [int(X_train.iyear.max()), int(X_valid.iyear.max()), int(X_test.
    ↪iyear.max())],
    "success_rate": [y_train.mean(), y_valid.mean(), y_test.mean()],
})

print("Data shape:", df.shape)

display(summary)
display(y.value_counts(normalize=True).rename("proportion").to_frame())

```

Loading cached cleaned data:

c:\Documents\CompSci\CSELEC2C\Project\data\processed\gtd_light_full.pkl

Saved processed temporal split files to:

c:\Documents\CompSci\CSELEC2C\Project\data\processed\temporal

Data shape: (214666, 28)

	split	n	start_year	end_year	success_rate
0	train	142325	1970	2014	0.909229
1	validation	40553	2015	2017	0.818706
2	test	31788	2018	2021	0.846829

	proportion
success	
1	0.882888
0	0.117112

0.4 4. Models

The proposed model is **GTD-FocalResNet**, a residual tabular neural network using focal loss, softened class weighting, feature dropout, and temperature scaling. It is compared against Logistic Regression, CART, XGBoost, and a standard Feedforward NN.

```

[17]: class TorchMLP(BaseEstimator, ClassifierMixin):
    def __init__(self, hidden=(128,64), residual=False, n_blocks=2,
    ↪focal=False, gamma=1.5, pos_power=0.6, feat_drop=0.05, temp=True, epochs=25,
    ↪lr=7e-4, wd=2e-4, batch=512, patience=3, min_delta=1e-4,
    ↪trim_history_to_best=True, use_gpu=True, verbose=True):
        self.hidden=hidden; self.residual=residual; self.n_blocks=n_blocks;
    ↪self.focal=focal; self.gamma=gamma; self.pos_power=pos_power; self.
    ↪feat_drop=feat_drop; self.temp=temp; self.epochs=epochs; self.lr=lr; self.
    ↪wd=wd; self.batch=batch; self.patience=patience; self.min_delta=min_delta;
    ↪self.trim_history_to_best=trim_history_to_best; self.use_gpu=use_gpu; self.
    ↪verbose=verbose

```

```

def _device(self): return torch.device("cuda" if self.use_gpu and torch is_
↳not None and torch.cuda.is_available() else "cpu")

def _tensor(self, X):
    if sparse.issparse(X): X = X.toarray()
    return torch.as_tensor(X, dtype=torch.float32, device=self.device_)

def _net(self, d):
    if not self.residual:
        layers=[]; prev=d
        for h in self.hidden: layers += [nn.Linear(prev,h), nn.ReLU(), nn.
↳Dropout(.30)]; prev=h
        return nn.Sequential(*layers, nn.Linear(prev,1))
    h = self.hidden[0]

    class ResNet(nn.Module):
        def __init__(self, d, h, n_blocks):
            super().__init__()
            self.inp = nn.Sequential(nn.Linear(d,h), nn.LayerNorm(h), nn.
↳GELU(), nn.Dropout(.20))
            self.blocks = nn.ModuleList([
                nn.Sequential(nn.Linear(h,h), nn.LayerNorm(h), nn.GELU(),
↳nn.Dropout(.20), nn.Linear(h,h), nn.LayerNorm(h))
                for _ in range(n_blocks)
            ])
            self.out = nn.Linear(h,1)
        def forward(self,x):
            z = self.inp(x)
            for block in self.blocks:
                z = torch.nn.functional.gelu(z + block(z))
            return self.out(z)
    return ResNet(d, h, self.n_blocks)

def _loss(self, logits, y, pos_w):
    bce = torch.nn.functional.binary_cross_entropy_with_logits(logits, y,
↳pos_weight=pos_w, reduction="none")
    if not self.focal: return bce.mean()
    p = torch.sigmoid(logits); pt = y*p + (1-y)*(1-p)

    return (((1-pt).clamp(min=1e-6)**self.gamma)*bce).mean()

def fit(self, X, y):
    if torch is None: raise ImportError("PyTorch required")

    torch.manual_seed(RANDOM_STATE); rng=np.random.
↳default_rng(RANDOM_STATE); self.classes_=np.array([0,1]); self.device_=self.
↳_device(); self.T_=1.0

```

```

        yy=np.asarray(y).astype(np.float32); self.net_=self._net(X.shape[1]).
        ↪to(self.device_)
        idx=np.arange(X.shape[0]); rng.shuffle(idx); v=max(int(.1*len(idx)),1);
        ↪vi,tr=idx[:v],idx[v:]
        pos=max((yy[tr]==1).sum(),1); neg=max((yy[tr]==0).sum(),1); pos_w=torch.
        ↪tensor([(neg/pos)**self.pos_power],device=self.device_)
        opt=torch.optim.AdamW(self.net_.parameters(),lr=self.
        ↪lr,weight_decay=self.wd); best=None; best_loss=1e9; best_epoch=0; stale=0

        # Fixed samples keep epoch-history plotting light and stable.
        hist_n = int(globals().get("HISTORY_SAMPLE_N", 20000))
        tr_hist = rng.choice(tr, size=min(hist_n, len(tr)), replace=False) if
        ↪len(tr) > hist_n else tr.copy()
        vi_hist = rng.choice(vi, size=min(hist_n, len(vi)), replace=False) if
        ↪len(vi) > hist_n else vi.copy()
        self.history_ = {"epoch": [], "train_loss": [], "validation_loss": [],
        ↪"train_accuracy": [], "validation_accuracy": [], "train_f1": [],
        ↪"validation_f1": [], "train_roc_auc": [], "validation_roc_auc": []}

        for ep in range(1,self.epochs+1):
            self.net_.train(); rng.shuffle(tr); losses=[]

            for s in range(0,len(tr),self.batch):
                b=tr[s:s+self.batch]; xb=self._tensor(X[b]); yb=torch.
                ↪as_tensor(yy[b],dtype=torch.float32,device=self.device_).view(-1,1)
                if self.feat_drop and self.residual:
                    keep=1-self.feat_drop; xb=xb*(torch.rand_like(xb)<keep).
                    ↪float()/keep
                opt.zero_grad(set_to_none=True); loss=self._loss(self.
                ↪net_(xb),yb,pos_w); loss.backward(); torch.nn.utils.clip_grad_norm_(self.
                ↪net_.parameters(),2.0); opt.step(); losses.append(float(loss.detach().cpu()))
                train_loss = float(np.mean(losses))
                vl=float(self._val_loss(X,yy,vi,pos_w))
                hist_train_loss, train_acc, train_f1, train_auc = self.
                ↪_metrics_on_indices(X, yy, tr_hist, pos_w)
                hist_val_loss, val_acc, val_f1, val_auc = self.
                ↪_metrics_on_indices(X, yy, vi_hist, pos_w)
                self.history_["epoch"].append(ep)
                self.history_["train_loss"].append(float(hist_train_loss))
                self.history_["validation_loss"].append(float(hist_val_loss))
                self.history_["train_accuracy"].append(float(train_acc))
                self.history_["validation_accuracy"].append(float(val_acc))
                self.history_["train_f1"].append(float(train_f1))
                self.history_["validation_f1"].append(float(val_f1))
                self.history_["train_roc_auc"].append(float(train_auc))
                self.history_["validation_roc_auc"].append(float(val_auc))

```

```

        if self.verbose and (ep==1 or ep%5==0): print(f"'GTD-FocalResNet'
→if self.residual else 'Feedforward NN'} epoch {ep:02d} | loss={train_loss:.
→4f} | val={vl:.4f} | acc={train_acc:.4f} | val_acc={val_acc:.4f}")
        if vl < best_loss - self.min_delta:
            best_loss=vl; best_epoch=ep; best={k:v.detach().cpu().clone()
→for k,v in self.net_.state_dict().items()}; stale=0
        else:
            stale+=1
            if stale>=self.patience: break

        self.best_epoch_ = best_epoch
        self.best_validation_loss_ = best_loss
        if best: self.net_.load_state_dict(best); self.net_.to(self.device_)
        if self.trim_history_to_best and best_epoch > 0:
            for key in self.history_:
                self.history_[key] = self.history_[key][:best_epoch]
        if self.temp: self._calibrate(X,yy,vi)

    return self

def _metrics_on_indices(self, X, yy, idxs, pos_w):
    if len(idxs) == 0: return np.nan, np.nan, np.nan, np.nan
    self.net_.eval(); losses=[]; probs=[]; truth=[]
    with torch.no_grad():
        for s in range(0, len(idxs), self.batch):
            b = idxs[s:s+self.batch]
            xb = self._tensor(X[b])
            yb = torch.as_tensor(yy[b], dtype=torch.float32, device=self.
→device_).view(-1,1)
            logits = self.net_(xb)
            losses.append(float(self._loss(logits, yb, pos_w).detach().
→cpu()))
            probs.extend(torch.sigmoid(logits).detach().cpu().numpy().
→ravel())
            truth.extend(yy[b])
        probs = np.asarray(probs)
        truth = np.asarray(truth).astype(int)
        pred = (probs >= 0.5).astype(int)
        auc = roc_auc_score(truth, probs) if len(np.unique(truth)) == 2 else np.
→nan
        return float(np.mean(losses)), float(accuracy_score(truth, pred)),
→float(f1_score(truth, pred, zero_division=0)), float(auc)

def _val_loss(self,X,yy,vi,pos_w):
    self.net_.eval(); vals=[]

```

```

        with torch.no_grad():
            for s in range(0, len(vi), self.batch):
                b=vi[s:s+self.batch]; vals.append(float(self._loss(self.
↪net_(self._tensor(X[b])), torch.as_tensor(yy[b], dtype=torch.
↪float32, device=self.device_).view(-1,1), pos_w).detach().cpu()))
            return np.mean(vals)

    def _logits(self, X):
        self.net_.eval(); out=[]
        with torch.no_grad():
            for s in range(0, X.shape[0], self.batch): out.extend(self.net_(self.
↪_tensor(X[s:s+self.batch])).detach().cpu().numpy().ravel())
        return np.asarray(out)

    def _calibrate(self, X, yy, vi):
        lg=self._logits(X[vi]); best=(1, 1e9)
        for T in np.linspace(.6, 2.5, 20):
            pp=1/(1+np.exp(-(lg/T))); br=np.mean((yy[vi]-pp)**2)
            if br<best[1]: best=(float(T), br)
        self.T_=best[0]

    def predict_proba(self, X):
        pp=1/(1+np.exp(-(self._logits(X)/self.T_))); return np.c_[1-pp, pp]

    def predict(self, X):
        return (self.predict_proba(X)[: , 1] >= 0.5).astype(int)

def make_models(nn_epochs=25): # Can ajust epochs for quick/heavy runs
    pp = preprocessor(num, cat)
    models = {
        "logistic_regression": Pipeline([("prep", pp), ("model",
↪LogisticRegression(max_iter=1500, class_weight="balanced", solver="saga",
↪random_state=RANDOM_STATE))]),
        "cart": Pipeline([("prep", preprocessor(num, cat)), ("model",
↪DecisionTreeClassifier(max_depth=12, min_samples_leaf=100,
↪min_samples_split=250, class_weight="balanced",
↪random_state=RANDOM_STATE))]),
        "feedforward_nn": Pipeline([("prep", preprocessor(num, cat)), ("model",
↪TorchMLP(hidden=(96, 48), residual=False, focal=False, temp=False,
↪epochs=nn_epochs, lr=.0006, wd=.0005, patience=3, min_delta=1e-4,
↪use_gpu=USE_GPU, verbose=True))]),
        "gtd_focal_resnet": Pipeline([("prep", preprocessor(num, cat)),
↪("model", TorchMLP(hidden=(160, ), residual=True, n_blocks=2, focal=True,
↪gamma=1.3, pos_power=.6, feat_drop=.08, temp=True, epochs=nn_epochs, lr=.
↪0005, wd=.0005, patience=3, min_delta=1e-4, use_gpu=USE_GPU,
↪verbose=True))]),

```

```

    }
    if XGBClassifier is not None:
        pos=max((y_train==1).sum(),1); neg=max((y_train==0).sum(),1);
        device="cuda" if USE_GPU and torch is not None and torch.cuda.is_available()
        else "cpu"
        models["xgboost"] = Pipeline([("prep", preprocessor(num,cat)),
        ("model",
        XGBClassifier(n_estimators=XGB_N_ESTIMATORS,max_depth=XGB_MAX_DEPTH,learning_rate=.
        05,subsample=.85,colsample_bytree=.85,objective="binary:
        logistic",eval_metric="logloss",scale_pos_weight=neg/
        pos,random_state=RANDOM_STATE,n_jobs=-1,tree_method="hist",device=device))])
        return {k:models[k] for k in
        ["logistic_regression","cart","xgboost","feedforward_nn","gtd_focal_resnet"]}
        if k in models}

def save_torch_pipeline(pipeline, pt_path):
    """Save a notebook-defined PyTorch pipeline without pickling the custom
    class.

    The sklearn preprocessor is saved separately with joblib, while the neural
    network weights and architecture settings are saved to a .pt checkpoint.
    """
    pt_path = Path(pt_path)
    prep_path = pt_path.with_name(pt_path.stem + "_preprocessor.joblib")
    prep = pipeline.named_steps["prep"]
    model = pipeline.named_steps["model"]

    joblib.dump(prepare, prep_path)
    checkpoint = {
        "model_params": {
            "hidden": model.hidden,
            "residual": model.residual,
            "n_blocks": model.n_blocks,
            "focal": model.focal,
            "gamma": model.gamma,
            "pos_power": model.pos_power,
            "feat_drop": model.feat_drop,
            "temp": model.temp,
            "epochs": model.epochs,
            "lr": model.lr,
            "wd": model.wd,
            "batch": model.batch,
            "patience": model.patience,
            "min_delta": model.min_delta,
            "trim_history_to_best": model.trim_history_to_best,
            "use_gpu": model.use_gpu,
            "verbose": False,

```



```

    },
    "input_dim": int(next(model.net_.parameters()).shape[1]) if not model.
↪residual else int(model.net_.inp[0].in_features),
    "temperature": float(getattr(model, "T_", 1.0)),
    "classes": getattr(model, "classes_", np.array([0, 1]).tolist()),
    "state_dict": {k: v.detach().cpu() for k, v in model.net_.state_dict().
↪items()},
    "history": getattr(model, "history_", None),
    "best_epoch": getattr(model, "best_epoch_", None),
    "best_validation_loss": getattr(model, "best_validation_loss_", None),
    "preprocessor_path": str(preprocessor.name),
}
torch.save(checkpoint, pt_path)
print(f"Saved PyTorch checkpoint: {pt_path}")
print(f"Saved matching preprocessor: {prep_path}")

def load_torch_pipeline(pt_path):
    """Reload a saved TorchMLP pipeline inside this notebook after running the_
↪model-definition cell."""
    pt_path = Path(pt_path)
    try:
        checkpoint = torch.load(pt_path, map_location="cpu", weights_only=False)
    except TypeError:
        checkpoint = torch.load(pt_path, map_location="cpu")
    prep_path = pt_path.with_name(checkpoint.get("preprocessor_path", pt_path.
↪stem + "_preprocessor.joblib"))
    prep = joblib.load(prepath)
    model = TorchMLP(**checkpoint["model_params"])
    model.classes_ = np.array(checkpoint.get("classes", [0, 1]))
    model.T_ = float(checkpoint.get("temperature", 1.0))
    model.device_ = model._device()
    model.net_ = model._net(int(checkpoint["input_dim"])).to(model.device_)
    model.net_.load_state_dict(checkpoint["state_dict"])
    model.net_.eval()
    return Pipeline([("prep", prep), ("model", model)])

```

0.5 5. Train models and report core scores

Only the requested scores are emphasized: accuracy, precision, recall, and F1. ROC-AUC is kept as a supporting ranking metric because it helps compare threshold-free behavior.

Saved models are written to `models/light_reproducible/<split_mode>/`. Sklearn/tree models are saved as `.joblib`. The two notebook-defined PyTorch models are saved as `.pt` checkpoints, with their fitted preprocessors saved beside them as `_preprocessor.joblib` files.

```
[18]: models = make_models(nn_epochs=MODEL_EPOCHS)

for name, model in models.items():
    print("Training", name)
    model.fit(X_train, y_train)

    # Save sklearn/tree models with joblib. Save notebook-defined PyTorch
    ↪models as .pt checkpoints.
    if name in {"logistic_regression", "cart", "xgboost"}:
        joblib.dump(model, MOD / f"{name}.joblib")
        print(f"Saved joblib model: {MOD / f'{name}.joblib'}")
    else:
        save_torch_pipeline(model, MOD / f"{name}.pt")

score_rows=[]

for split, Xs, ys in [("validation",X_valid,y_valid),("test",X_test,y_test)]:
    for name, model in models.items():
        score_rows.append(scores(name, split, ys, proba(model, Xs)))

scores_df=pd.DataFrame(score_rows)
scores_df.to_csv(TAB / "scores.csv", index=False)

display(scores_df)
```

```
Training logistic_regression
Saved joblib model: c:\Documents\Compisci\CSELEC2C\Project\models\light_reproduc
ible\temporal\logistic_regression.joblib
Training cart
Saved joblib model: c:\Documents\Compisci\CSELEC2C\Project\models\light_reproduc
ible\temporal\cart.joblib
Training xgboost
Saved joblib model: c:\Documents\Compisci\CSELEC2C\Project\models\light_reproduc
ible\temporal\xgboost.joblib
Training feedforward_nn
Feedforward NN epoch 01 | loss=0.1663 | val=0.1473 | acc=0.8746 | val_acc=0.8751
Feedforward NN epoch 05 | loss=0.1356 | val=0.1365 | acc=0.8931 | val_acc=0.8944
Feedforward NN epoch 10 | loss=0.1290 | val=0.1356 | acc=0.8962 | val_acc=0.8908
Saved PyTorch checkpoint: c:\Documents\Compisci\CSELEC2C\Project\models\light_rep
roducible\temporal\feedforward_nn.pt
Saved matching preprocessor: c:\Documents\Compisci\CSELEC2C\Project\models\light_
reproducible\temporal\feedforward_nn_preprocessor.joblib
Training gtd_focal_resnet
GTD-FocalResNet epoch 01 | loss=0.0659 | val=0.0587 | acc=0.8881 |
val_acc=0.8900
GTD-FocalResNet epoch 05 | loss=0.0559 | val=0.0570 | acc=0.9113 |
val_acc=0.9073
```

GTD-FocalResNet epoch 10 | loss=0.0528 | val=0.0565 | acc=0.9094 |
val_acc=0.9020
Saved PyTorch checkpoint: c:\Documents\Compisci\CSELEC2C\Project\models\light_reproducible\temporal\gtd_focal_resnet.pt
Saved matching preprocessor: c:\Documents\Compisci\CSELEC2C\Project\models\light_reproducible\temporal\gtd_focal_resnet_preprocessor.joblib

	model	split	accuracy	precision	recall	f1	\
0	logistic_regression	validation	0.751313	0.907287	0.775489	0.836227	
1	cart	validation	0.716125	0.899209	0.735731	0.809297	
2	xgboost	validation	0.794121	0.912276	0.828168	0.868190	
3	feedforward_nn	validation	0.839247	0.901571	0.902142	0.901856	
4	gtd_focal_resnet	validation	0.835623	0.898555	0.900937	0.899744	
5	logistic_regression	test	0.748364	0.916557	0.773246	0.838824	
6	cart	test	0.741632	0.903773	0.777703	0.836012	
7	xgboost	test	0.800365	0.916542	0.840819	0.877049	
8	feedforward_nn	test	0.829244	0.908652	0.887589	0.897997	
9	gtd_focal_resnet	test	0.826507	0.907075	0.885880	0.896352	

roc_auc

0	0.780362
1	0.767209
2	0.824982
3	0.837914
4	0.835503
5	0.769711
6	0.741202
7	0.809079
8	0.804565
9	0.797238

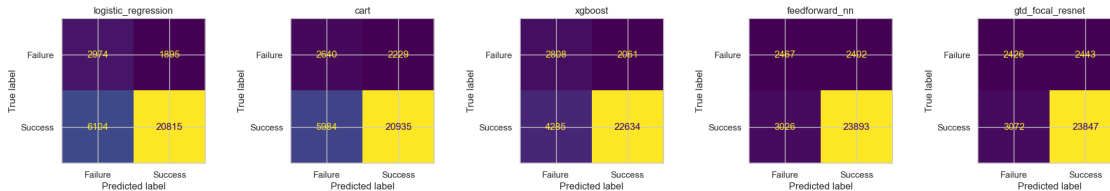
0.6 6. Confusion matrices

The confusion matrices use the default 0.50 threshold. This makes the comparison direct and easy to reproduce.

```
[19]: fig, axes = plt.subplots(1, len(models), figsize=(4*len(models), 3.5))

if len(models)==1: axes=[axes]

for ax, (name, model) in zip(axes, models.items()):
    pred=(proba(model, X_test)>=.5).astype(int)
    ConfusionMatrixDisplay(confusion_matrix(y_test, pred),
        display_labels=["Failure", "Success"]).plot(ax=ax, colorbar=False)
    ax.set_title(name)
plt.tight_layout(); plt.savefig(FIG / "confusion_matrices.png", dpi=180); plt.
    show()
```



0.7 7. Epoch learning curves for neural models

The neural models now use stronger regularization and stricter early stopping. The displayed epoch curves are trimmed to the best validation-loss checkpoint, which avoids showing extra epochs after validation performance has already started to degrade. A small rolling mean is used only for display; the raw epoch values are still saved in the CSV output. The first figure shows accuracy and loss, while the second figure adds F1 and ROC-AUC curves.

```
[20]: if RUN_LEARNING_CURVES:
    history_rows = []
    neural_models = [
        ("feedforward_nn", "Feedforward NN"),
        ("gtd_focal_resnet", "GTD-FocalResNet"),
    ]
    smooth_window = int(globals().get("HISTORY_SMOOTH_WINDOW", 3))

    def smooth(series):
        if smooth_window <= 1 or len(series) < smooth_window:
            return series
        return series.rolling(window=smooth_window, min_periods=1).mean()

    def plot_pair(df, label, metric_left, metric_right, ylabel_left,
        ylabel_right, title_left, title_right, output_name, legend_left="lower",
        legend_right="upper right"):
        fig, axes = plt.subplots(1, 2, figsize=(12, 3.6))
        x = df["epoch"]
        for ax, metric, ylabel, title, loc in [
            (axes[0], metric_left, ylabel_left, title_left, legend_left),
            (axes[1], metric_right, ylabel_right, title_right, legend_right),
        ]:
            train_col = f"train_{metric}"
            val_col = f"validation_{metric}"
            ax.plot(x, df[train_col], color="tab:blue", alpha=0.20)
            ax.plot(x, df[val_col], color="tab:orange", alpha=0.20)
            ax.plot(x, smooth(df[train_col]), color="tab:blue", label=f"Train_{ylabel}")
            ax.plot(x, smooth(df[val_col]), color="tab:orange", label=f"Validation {ylabel}")
            ax.set_title(f"{label} - {title}")
```

```

        ax.set_xlabel("Epoch")
        ax.set_ylabel(ylabel)
        ax.grid(True, alpha=0.8)
        ax.legend(loc=loc)
    plt.tight_layout()
    plt.savefig(FIG / output_name, dpi=180, bbox_inches="tight")
    plt.show()

for name, label in neural_models:
    if name not in models:
        continue
    torch_model = models[name].named_steps["model"]
    hist = getattr(torch_model, "history_", None)
    if not hist:
        print(f"No epoch history found for {name}. Rerun the training cell_
↳after the patched TorchMLP definition.")
        continue
    best_epoch = getattr(torch_model, "best_epoch_", None)
    best_loss = getattr(torch_model, "best_validation_loss_", None)
    if best_epoch is not None:
        print(f"{label}: using best validation-loss checkpoint from epoch_
↳{best_epoch} (val_loss={best_loss:.4f}).")
    hist_df_model = pd.DataFrame(hist)
    hist_df_model["model"] = name
    history_rows.append(hist_df_model)

    plot_pair(
        hist_df_model, label,
        metric_left="accuracy", metric_right="loss",
        ylabel_left="Accuracy", ylabel_right="Loss",
        title_left="Accuracy", title_right="Loss",
        output_name=f"learning_curve_{name}_accuracy_loss.png",
        legend_left="lower right", legend_right="upper right"
    )
    plot_pair(
        hist_df_model, label,
        metric_left="f1", metric_right="roc_auc",
        ylabel_left="F1", ylabel_right="ROC-AUC",
        title_left="F1", title_right="ROC-AUC",
        output_name=f"learning_curve_{name}_f1_roc_auc.png",
        legend_left="lower right", legend_right="lower right"
    )

if history_rows:
    epoch_lc_df = pd.concat(history_rows, ignore_index=True)
    epoch_lc_df.to_csv(TAB / "learning_curves_epoch_neural_models.csv",
↳index=False)

```

```

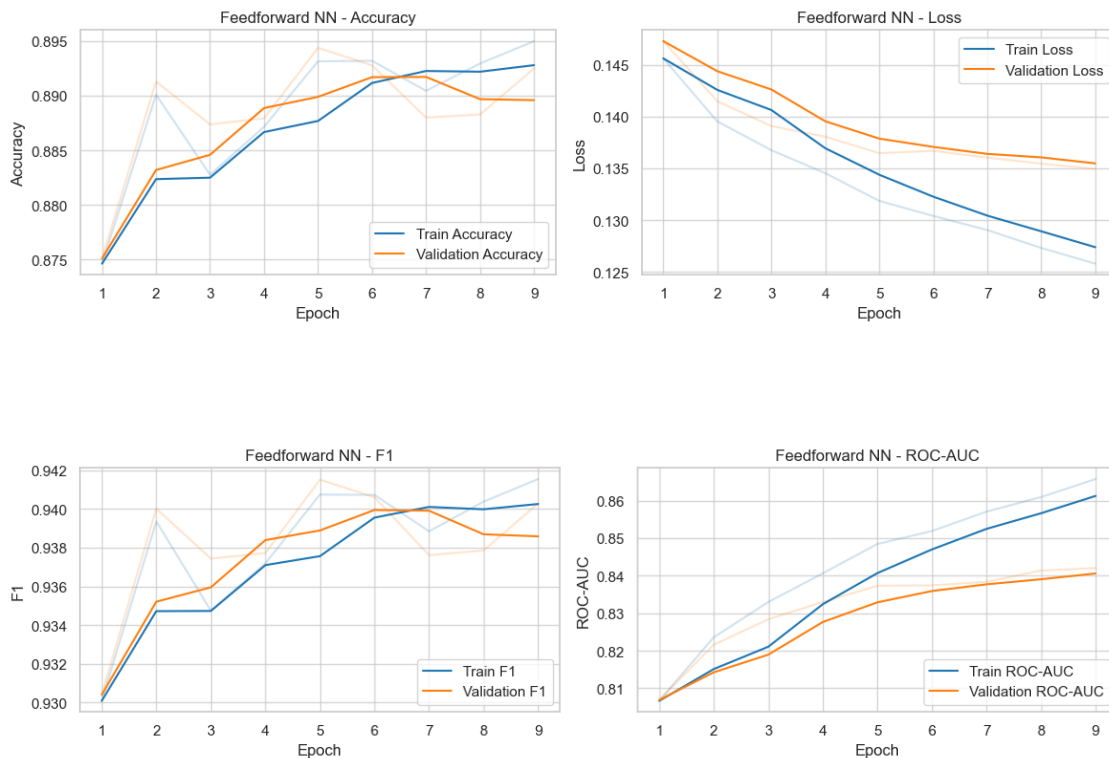
# Backward-compatible filename used by older output summaries.
epoch_lc_df.to_csv(TAB /
↳"learning_curves_epoch_accuracy_loss_neural_models.csv", index=False)

for metric_left, metric_right, ylabel_left, ylabel_right, title_left,
↳title_right, filename in [
    ("accuracy", "loss", "Accuracy", "Loss", "Accuracy", "Loss",
↳"learning_curve_neural_models_accuracy_loss.png"),
    ("f1", "roc_auc", "F1", "ROC-AUC", "F1", "ROC-AUC",
↳"learning_curve_neural_models_f1_roc_auc.png"),
]:
    fig, axes = plt.subplots(1, 2, figsize=(12, 3.8))
    for name, label in neural_models:
        sub = epoch_lc_df[epoch_lc_df["model"] == name]
        if sub.empty:
            continue
        for ax, metric, ylabel, title in [
            (axes[0], metric_left, ylabel_left, title_left),
            (axes[1], metric_right, ylabel_right, title_right),
        ]:
            ax.plot(sub["epoch"], smooth(sub[f"train_{metric}"]),
↳linestyle="--", label=f"{label} Train")
            ax.plot(sub["epoch"], smooth(sub[f"validation_{metric}"]),
↳label=f"{label} Validation")
            ax.set_title(f"Neural Models - {title}")
            ax.set_xlabel("Epoch")
            ax.set_ylabel(ylabel)
            ax.grid(True, alpha=0.8)
            ax.legend(fontsize=8)
        plt.tight_layout()
        plt.savefig(FIG / filename, dpi=180, bbox_inches="tight")
        plt.show()

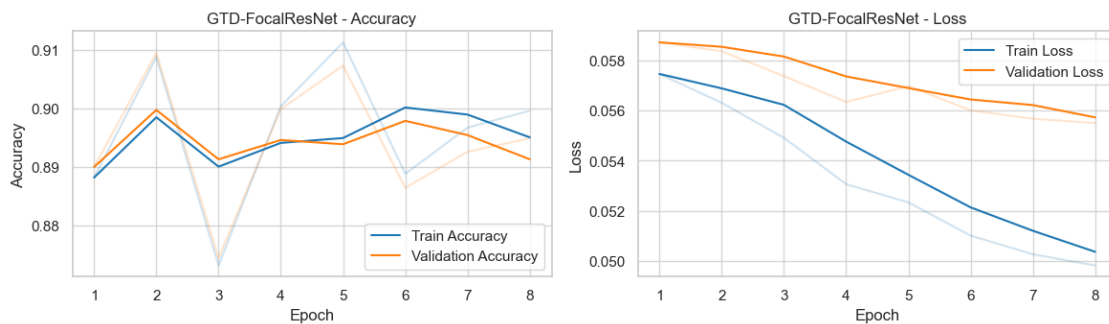
display(epoch_lc_df[["model", "epoch", "train_accuracy",
↳"validation_accuracy", "train_loss", "validation_loss", "train_f1",
↳"validation_f1", "train_roc_auc", "validation_roc_auc"]])

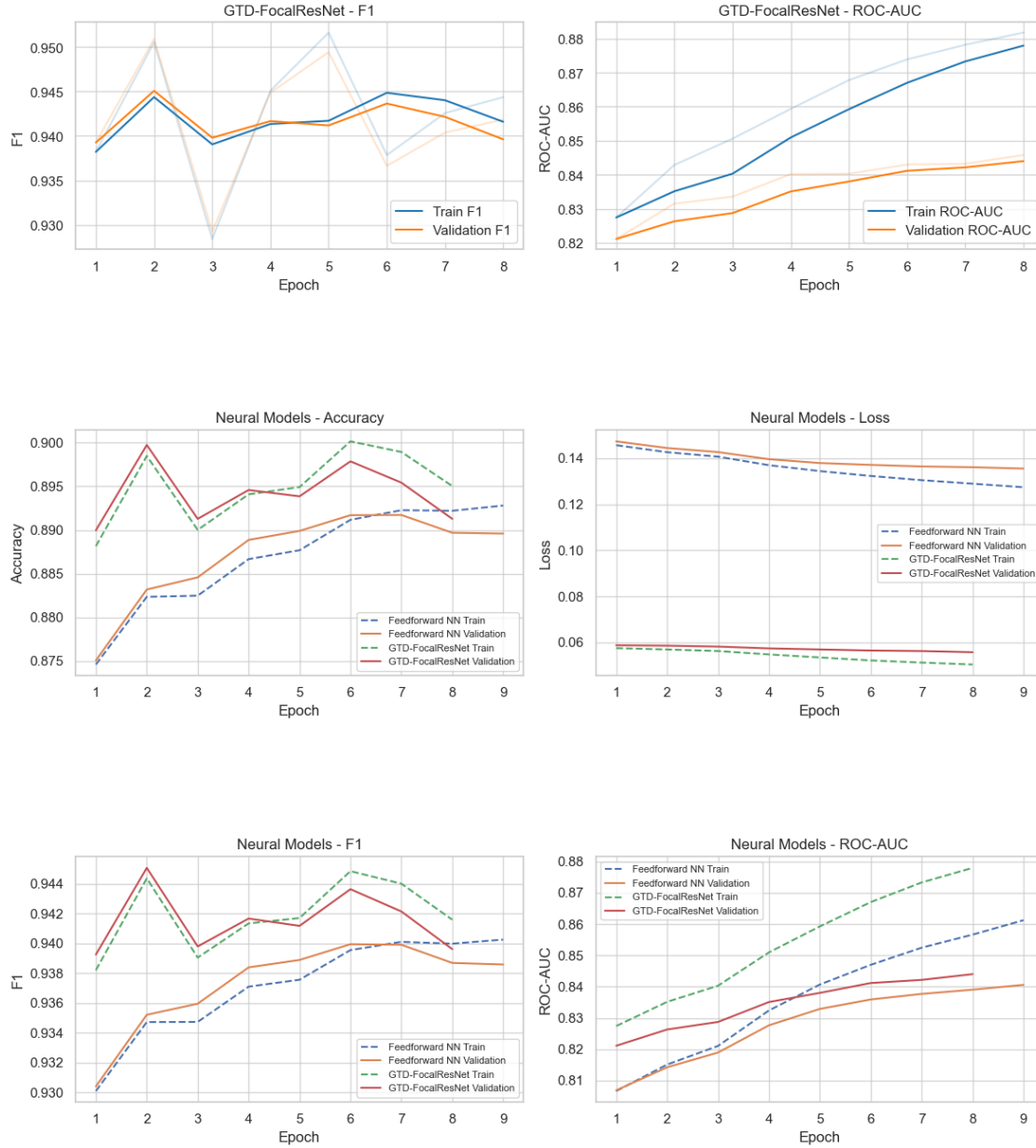
```

Feedforward NN: using best validation-loss checkpoint from epoch 9
(val_loss=0.1350).



GTD-FocalResNet: using best validation-loss checkpoint from epoch 8 (val_loss=0.0555).





	model	epoch	train_accuracy	validation_accuracy	train_loss	\
0	feedforward_nn	1	0.87460	0.875070	0.145636	
1	feedforward_nn	2	0.89010	0.891301	0.139540	
2	feedforward_nn	3	0.88275	0.887366	0.136763	
3	feedforward_nn	4	0.88715	0.887929	0.134535	
4	feedforward_nn	5	0.89315	0.894393	0.131853	
5	feedforward_nn	6	0.89320	0.892777	0.130400	
6	feedforward_nn	7	0.89045	0.887999	0.129040	
7	feedforward_nn	8	0.89295	0.888280	0.127304	
8	feedforward_nn	9	0.89500	0.892496	0.125787	

9	gtd_focal_resnet	1	0.88815	0.889966	0.057454
10	gtd_focal_resnet	2	0.90880	0.909500	0.056303
11	gtd_focal_resnet	3	0.87310	0.874368	0.054907
12	gtd_focal_resnet	4	0.90035	0.899874	0.053059
13	gtd_focal_resnet	5	0.91130	0.907322	0.052333
14	gtd_focal_resnet	6	0.88880	0.886383	0.051008
15	gtd_focal_resnet	7	0.89670	0.892566	0.050262
16	gtd_focal_resnet	8	0.89960	0.894885	0.049818

	validation_loss	train_f1	validation_f1	train_roc_auc	\
0	0.147301	0.930081	0.930395	0.806650	
1	0.141497	0.939355	0.940018	0.823587	
2	0.139099	0.934747	0.937434	0.832971	
3	0.138067	0.937189	0.937722	0.840632	
4	0.136468	0.940746	0.941515	0.848436	
5	0.136699	0.940726	0.940604	0.851884	
6	0.136041	0.938845	0.937613	0.857051	
7	0.135454	0.940380	0.937857	0.860928	
8	0.134957	0.941547	0.940286	0.865725	
9	0.058713	0.938203	0.939251	0.827420	
10	0.058361	0.950542	0.950903	0.842874	
11	0.057367	0.928402	0.929244	0.850611	
12	0.056338	0.945092	0.944868	0.859352	
13	0.056965	0.951628	0.949431	0.867810	
14	0.056005	0.937849	0.936635	0.873884	
15	0.055668	0.942570	0.940378	0.878193	
16	0.055497	0.944367	0.941835	0.881762	

	validation_roc_auc
0	0.806887
1	0.821610
2	0.828376
3	0.832987
4	0.837301
5	0.837420
6	0.838336
7	0.841359
8	0.842029
9	0.821117
10	0.831516
11	0.833554
12	0.840193
13	0.840289
14	0.842985
15	0.843227
16	0.845789

0.8 8. Permutation feature importance for all models

Permutation importance is used for every model so the importance method is consistent across linear, tree, boosted-tree, and neural models. It uses F1 on a fixed test sample, so the cell may take a few minutes when all models are enabled.

```
[21]: if RUN_FEATURE_IMPORTANCE:
    X_imp, y_imp = X_test, y_test

    if len(X_imp) > PERMUTATION_SAMPLE_N:
        X_imp, _, y_imp, _ = train_test_split(X_imp, y_imp,
        ↪train_size=PERMUTATION_SAMPLE_N, stratify=y_imp, random_state=RANDOM_STATE)

    imp_rows=[]

    for name, model in models.items():
        print("Permutation importance:", name)
        pi = permutation_importance(model, X_imp, y_imp, scoring="f1",
        ↪n_repeats=N_PERM_REPEATS, random_state=RANDOM_STATE, n_jobs=1)
        part = pd.DataFrame({"model": name, "feature": X_imp.columns,
        ↪"importance_mean": pi.importances_mean, "importance_std": pi.
        ↪importances_std})
        imp_rows.append(part)

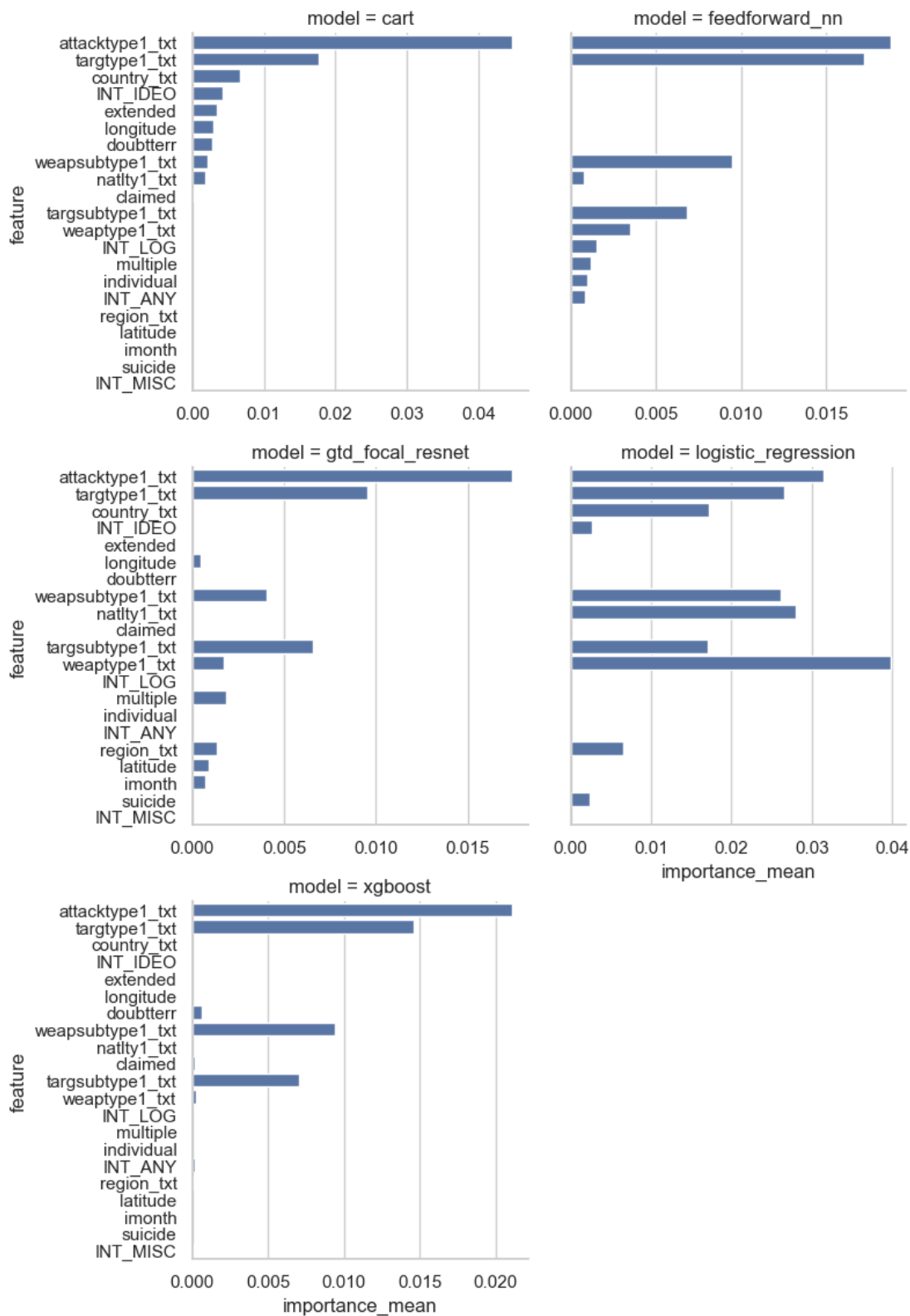
    imp_df=pd.concat(imp_rows, ignore_index=True).
    ↪sort_values(["model", "importance_mean"], ascending=[True, False])
    imp_df.to_csv(TAB / "permutation_feature_importance_all_models.csv",
    ↪index=False)
    top = imp_df.groupby("model").head(10)
    g=sns.catplot(data=top, y="feature", x="importance_mean", col="model",
    ↪kind="bar", col_wrap=2, sharex=False, height=4)
    g.fig.suptitle("Permutation Feature Importance by Model", y=1.02)

    plt.tight_layout(); plt.savefig(FIG / "feature_importance_all_models.png",
    ↪dpi=180); plt.show()

    display(top)
```

```
Permutation importance: logistic_regression
Permutation importance: cart
Permutation importance: xgboost
Permutation importance: feedforward_nn
Permutation importance: gtd_focal_resnet
```

Permutation Feature Importance by Model



	model	feature	importance_mean	importance_std
46	cart	attacktype1_txt	0.044616	0.001125
47	cart	targetype1_txt	0.017676	0.000996
45	cart	country_txt	0.006631	0.001554
41	cart	INT_IDEO	0.004240	0.000867
29	cart	extended	0.003356	0.000077
31	cart	longitude	0.002948	0.001156
34	cart	doubtterr	0.002789	0.000412
51	cart	weapsubtype1_txt	0.002158	0.000366
49	cart	natlty1_txt	0.001897	0.000090
39	cart	claimed	0.000281	0.000133
98	feedforward_nn	attacktype1_txt	0.018753	0.001684
99	feedforward_nn	targetype1_txt	0.017232	0.000387
103	feedforward_nn	weapsubtype1_txt	0.009457	0.000447
100	feedforward_nn	targsubtype1_txt	0.006842	0.001577
102	feedforward_nn	weaptype1_txt	0.003513	0.000712
92	feedforward_nn	INT_LOG	0.001529	0.000301
87	feedforward_nn	multiple	0.001180	0.000491
90	feedforward_nn	individual	0.000991	0.000109
95	feedforward_nn	INT_ANY	0.000858	0.000287
101	feedforward_nn	natlty1_txt	0.000752	0.000922
124	gtd_focal_resnet	attacktype1_txt	0.017387	0.002170
125	gtd_focal_resnet	targetype1_txt	0.009541	0.000754
126	gtd_focal_resnet	targsubtype1_txt	0.006582	0.001085
129	gtd_focal_resnet	weapsubtype1_txt	0.004062	0.000412
113	gtd_focal_resnet	multiple	0.001864	0.000728
128	gtd_focal_resnet	weaptype1_txt	0.001713	0.001133
122	gtd_focal_resnet	region_txt	0.001337	0.000793
108	gtd_focal_resnet	latitude	0.000867	0.000326
105	gtd_focal_resnet	imonth	0.000696	0.000372
109	gtd_focal_resnet	longitude	0.000432	0.000149
24	logistic_regression	weaptype1_txt	0.039753	0.002430
20	logistic_regression	attacktype1_txt	0.031482	0.001489
23	logistic_regression	natlty1_txt	0.027936	0.001530
21	logistic_regression	targetype1_txt	0.026506	0.001718
25	logistic_regression	weapsubtype1_txt	0.026141	0.002346
19	logistic_regression	country_txt	0.017238	0.000798
22	logistic_regression	targsubtype1_txt	0.017028	0.002401
18	logistic_regression	region_txt	0.006478	0.001209
15	logistic_regression	INT_IDEO	0.002635	0.000929
10	logistic_regression	suicide	0.002300	0.000215
72	xgboost	attacktype1_txt	0.021076	0.001442
73	xgboost	targetype1_txt	0.014583	0.001043
77	xgboost	weapsubtype1_txt	0.009423	0.000687
74	xgboost	targsubtype1_txt	0.007051	0.001132

60	xgboost	doubtterr	0.000636	0.000865
76	xgboost	weaptype1_txt	0.000248	0.000449
65	xgboost	claimed	0.000182	0.000483
69	xgboost	INT_ANY	0.000159	0.000262
68	xgboost	INT_MISC	0.000097	0.000071
70	xgboost	region_txt	0.000088	0.000625

0.9 9. Output summary

Outputs are saved under `results/light_reproducible/<split_mode>/`. Sklearn/tree models are saved as `.joblib` under `models/light_reproducible/<split_mode>/`; notebook-defined Torch models are saved there as `.pt` checkpoints with matching fitted preprocessors.

```
[22]: print("Scores:", TAB / "scores.csv")
print("Confusion matrices:", FIG / "confusion_matrices.png")
print("Learning curves:", TAB / "learning_curves_epoch_neural_models.csv", FIG /
      ↪ "learning_curve_neural_models_accuracy_loss.png", FIG /
      ↪ "learning_curve_neural_models_f1_roc_auc.png")
print("Feature importance:", TAB / "permutation_feature_importance_all_models.
      ↪ csv", FIG / "feature_importance_all_models.png")
```

```
Scores: c:\Documents\CompSci\CSELEC2C\Project\results\light_reproducible\temporal\tables\scores.csv
Confusion matrices: c:\Documents\CompSci\CSELEC2C\Project\results\light_reproducible\temporal\figures\confusion_matrices.png
Learning curves: c:\Documents\CompSci\CSELEC2C\Project\results\light_reproducible\temporal\tables\learning_curves_epoch_neural_models.csv c:\Documents\CompSci\CSELEC2C\Project\results\light_reproducible\temporal\figures\learning_curve_neural_models_accuracy_loss.png c:\Documents\CompSci\CSELEC2C\Project\results\light_reproducible\temporal\figures\learning_curve_neural_models_f1_roc_auc.png
Feature importance: c:\Documents\CompSci\CSELEC2C\Project\results\light_reproducible\temporal\tables\permutation_feature_importance_all_models.csv c:\Documents\CompSci\CSELEC2C\Project\results\light_reproducible\temporal\figures\feature_importance_all_models.png
```